

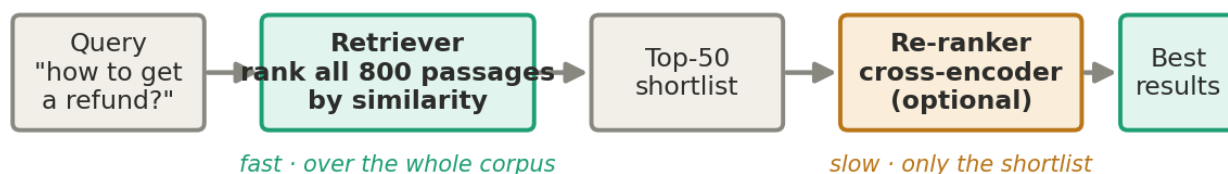
Key Learnings — Text Retrieval

ISBA 2411 · Natural Language Processing & AI · Week 4, Lecture 7 · The Retrieval Hackathon

The big idea: keyword search matches words; semantic search matches meaning. Neither wins everywhere — the best system fuses them, and the right choice depends on the query and your cost budget. This sheet is your takeaway from the hackathon: the methods you used, how they work, what they scored, and where to read more.

How retrieval works

The retrieval pipeline



Retrieval is a two-step idea: a fast **retriever** ranks the whole corpus by similarity to your query, and an optional **re-ranker** re-scores just the top handful with a heavier, more accurate model. You measure quality with **MRR@10** (1.0 if the right passage is #1, 0.5 if #2, 0 if it is not in the top 10) and **Recall@5** (is it in the top 5?), traded off against **cost** (build time, latency, size).

Two ways to represent text

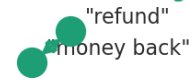
Sparse — keyword (TF-IDF, BM25)

	money	back	refund	policy	weather
query "money back"	1	1	0	0	0
passage "refund policy"	0	0	1	1	0

no shared words → similarity 0 → MISS

Dense — embeddings (meaning)

close in vector space
→ same meaning → MATCH

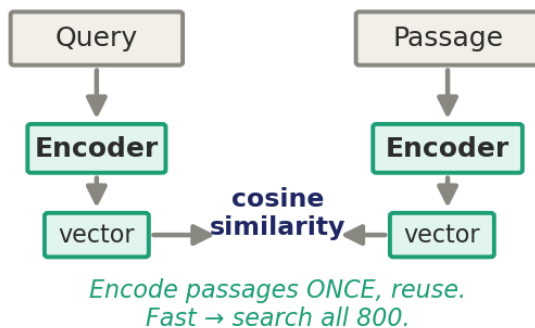


Sparse (keyword) — TF-IDF, BM25. Each document is a long vector of word counts, weighted so rare words matter more. Fast, interpretable, and excellent when the query shares words with the answer — but blind to paraphrases: "money back" and "refund" share no words, so similarity is zero.

Dense (embeddings). A neural model maps text to a short vector of continuous numbers so that *meaning*-similar texts sit close together in space. "money back" lands near "refund" even with no shared words. This is what powers semantic search.

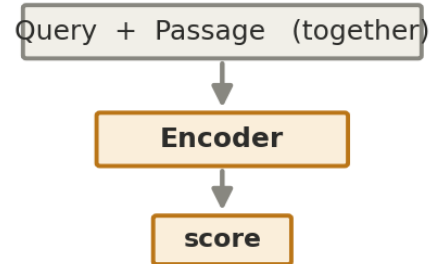
Two ways to use a neural model

Bi-encoder — used to RETRIEVE



Bi-encoder (to retrieve). Encodes the query and each passage *separately* into vectors, then compares by cosine similarity. You encode all 800 passages once and reuse them, so it is fast enough to search the whole corpus. Models: MiniLM, mpnet, E5.

Cross-encoder — used to RE-RANK



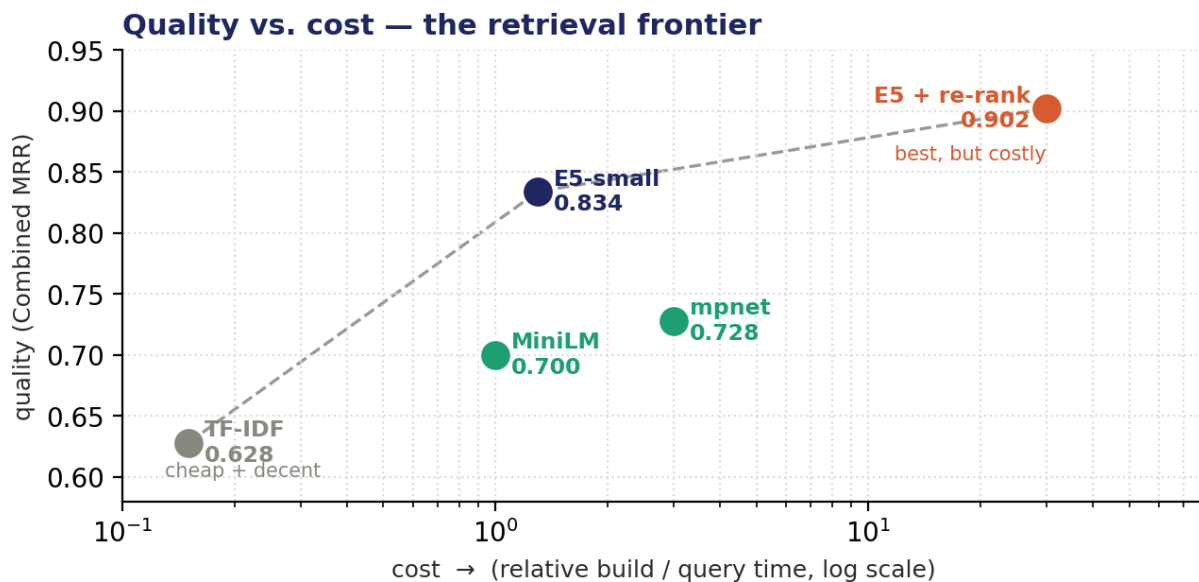
Reads both at once → more accurate.
Too slow for 800 → only the top ~50.

Cross-encoder (to re-rank). Feeds the query and a passage *together* through one model that outputs a single relevance score. It reads both at once, so it is more accurate — but far too slow to run on all 800. The trick: retrieve a shortlist with a bi-encoder, then re-rank the top ~50 with the cross-encoder. In the hackathon this was the single biggest jump in quality.

The methods you used — and what they scored

Method	Type	Main	Curve	Combined	Notes
TF-IDF	sparse keyword	0.782	0.474	0.628	the baseline
BM25	sparse keyword	~0.74	~0.39	—	classic keyword ranker
MiniLM	dense bi-encoder	0.737	0.664	0.700	fast, small
mpnet	dense bi-encoder	0.778	0.677	0.728	more accurate, slower
E5-small	dense bi-encoder	0.846	0.822	0.834	strongest single (needs query:/passage: prefixes)
E5 + cross-encoder	re-rank	0.937	0.868	0.902	the ceiling

Higher MRR = the correct passage sits nearer the top. **Combined = (Main + Curve) / 2**. Numbers validated on this dataset (800 passages).



Seven things to take away

- Keyword is a strong baseline.** On literal questions, TF-IDF/BM25 match or beat neural models — and cost almost nothing. Don't skip the baseline.
- Semantic search wins on paraphrases.** When the query and answer use different words (the curveball round), embeddings pull ahead. That gap is the whole reason they exist.
- Model choice can beat fancy fusion.** A top retrieval model (E5-small) outscores a hybrid of weaker ones. Pick a good model before over-engineering.
- Retrieve fast, then re-rank.** A bi-encoder shortlist + a cross-encoder re-rank was the biggest single quality jump (0.83 → 0.90 combined).
- Use each model correctly.** E5 needs query : / passage : prefixes; skip them and it underperforms. Read the model card.
- Tune on a dev set, not the test.** Picking settings by maximizing the leaderboard queries is the train-on-test trap — it won't generalize.
- It's a frontier, not a winner.** More quality usually costs time, size, or a GPU. The right model is the cheapest one that's good enough for the job.

Where this goes next: retrieval is the **R in RAG** (retrieval-augmented generation, Week 8) and a candidate representation for your team project.

Read more

Vector semantics & TF-IDF. Jurafsky & Martin, *Speech and Language Processing*, ch. 6 · <https://web.stanford.edu/~jurafsky/slp3/6.pdf>

Semantic search & RAG. Alammam & Grootendorst, *Hands-On Large Language Models*, ch. 8 · <https://www.oreilly.com/library/view/hands-on-large-language/9781098150952/>

BM25 (keyword ranking). Okapi BM25 — overview · https://en.wikipedia.org/wiki/Okapi_BM25

Sentence embeddings (bi-encoder). Reimers & Gurevych, *Sentence-BERT* (2019) · <https://arxiv.org/abs/1908.10084>

Sentence-Transformers library & retrieve-then-rerank guide. sbert.net documentation · https://www.sbert.net/examples/applications/retrieve_rerank/README.html

MiniLM. Wang et al. (2020) · <https://arxiv.org/abs/2002.10957>

MPNet. Song et al. (2020) · <https://arxiv.org/abs/2004.09297>

E5 embeddings. Wang et al., *Text Embeddings by Weakly-Supervised Contrastive Pre-training* (2022) · <https://arxiv.org/abs/2212.03533>

Compare embedding models. MTEB leaderboard · <https://huggingface.co/spaces/mteb/leaderboard>

MRR (the metric). Mean Reciprocal Rank · https://en.wikipedia.org/wiki/Mean_reciprocal_rank

Model cards (Hugging Face): [all-MiniLM-L6-v2](#) · [all-mpnet-base-v2](#) · [intfloat/e5-small-v2](#) · [cross-encoder/ms-marco-MiniLM-L-6-v2](#)